

Escribir Código Pythonico

¿Qué significa realmente? Ser 'pythonico' implica seguir las filosofías de diseño esenciales de Python, priorizando la legibilidad, la claridad y la simplicidad sobre la complejidad innecesaria. Esta guía resume las mejores prácticas basadas en la PEP 8 y el Zen de Python.

1. El Zen de Python

Si ejecutas `import this` en tu terminal, verás los principios que guían el desarrollo del lenguaje. Los tres pilares fundamentales son:

- **Bello es mejor que feo:** El código debe ser limpio y agradable a la vista.
- **Explícito es mejor que implícito:** No dejes que Python adivine tus intenciones; sé claro.
- **Simple es mejor que complejo:** Busca siempre la solución directa antes de inventar estructuras complicadas.

2. Formateo Esencial según PEP 8

La consistencia visual es clave para que cualquier programador pueda entender tu código rápidamente.

- **Sangría (Indentation):** Usa siempre 4 espacios por cada nivel de bloque. Nunca uses tabuladores.
- **Longitud de línea:** Limita las líneas de código a un máximo de 79 caracteres.
- **Líneas en blanco:** Separa las funciones globales y clases con dos líneas en blanco. Los métodos dentro de clases se separan con una sola línea.

3. Ejemplos Prácticos: No Pythonico vs Pythonico

A continuación, se presentan patrones comunes y cómo transformarlos en código limpio y eficiente:

Enfoque Incorrecto / Complejo	Solución Pythonica
-------------------------------	--------------------

<pre># Malo: Usar range e índices for i in range(len(lista)): print(i, lista[i])</pre>	<pre># Pythonico: Usar enumerate() for i, item in enumerate(lista): print(i, item)</pre>
<pre># Malo: Bucles largos para listas cuadrados = [] for x in range(10): cuadrados.append(x**2)</pre>	<pre># Pythonico: List Comprehension cuadrados = [x**2 for x in range(10)]</pre>
<pre># Malo: Variable temporal temp = a a = b b = temp</pre>	<pre># Pythonico: Asignación múltiple a, b = b, a</pre>
<pre># Malo: Comparar longitud if len(mi_lista) == 0: pass</pre>	<pre># Pythonico: Veracidad implícita if not mi_lista: pass</pre>

4. Reglas de Nomenclatura

El nombre de tus variables y funciones debe indicar claramente qué hacen:

- **snake_case:** Se usa para funciones, variables, atributos y métodos (ej. `calcular_promedio()`).
- **PascalCase:** Se usa exclusivamente para nombres de clases (ej. `UsuarioAdministrador`).
- **UPPER_CASE:** Se usa para definir constantes que nunca cambian su valor (ej. `PI = 3.1416`).